



UNIVERSITÀ DEGLI STUDI DI FIRENZE

SCUOLA DI INGEGNERIA – DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Tesi di Laurea Triennale in Ingegneria Informatica

MONITORAGGIO AMBIENTALE MEDIANTE SQUADRA DI ROBOT MOBILI

Candidato

Leonardo Soldani

Relatore

Prof. Giorgio Battistelli

Anno Accademico 2025/2026

Indice

1	Introduzione	1
2	Mappa, Agenti e Meccanismi di Coordinamento	3
2.1	Mappa e griglia di copertura	3
2.2	Agenti e algoritmo di pianificazione	4
2.2.1	Architettura dell'agente	5
2.3	Semplificazioni del modello	6
2.4	Struttura della simulazione	7
2.4.1	Parametrizzazione	7
2.4.2	Ciclo operativo	8
3	Implementazione	10
3.1	Tecnologie e ambiente di sviluppo	10
3.2	Architettura software	11
3.3	Rappresentazione della mappa	13
3.4	Logica dell'agente	15
3.5	Algoritmi di supporto	18
3.5.1	Modulo <code>bresenham_utilities.py</code>	18
3.5.2	Modulo <code>spawn_utilities.py</code>	20
3.6	Gestione del ciclo di simulazione	21

3.7	Post-processing dei dati	22
3.7.1	Elaborazione statistica dei dati	23
3.7.2	Analisi del singolo scenario	24
3.7.3	Analisi comparativa multi-run	26
4	Risultati sperimentali	27
4.1	Analisi temporale del valore di copertura	29
4.2	Distribuzione spaziale del valore di copertura	31
4.3	Analisi visiva e comportamento dinamico della flotta	34
4.4	Analisi parametrica	37
4.4.1	Dimensione della flotta	38
4.4.2	Campionamento non vincolato	40
4.4.3	Velocità di navigazione	41
4.4.4	Raggio di spawn	43
5	Analisi critica del modello	45
5.1	Limiti del modello	45
5.2	Sviluppi futuri	47
6	Conclusioni	49
	Bibliografia	52

Capitolo 1

Introduzione

La salvaguardia e il monitoraggio degli ecosistemi marini e costieri rappresentano da anni uno dei temi più sensibili nelle agende di ricerca nazionale ed europea, come evidenziato dalle priorità del Programma Nazionale di Ricerca (PNR 2021-2027) e dalle direttive del programma Horizon Europe. In questo scenario di frontiera scientifica si collocano iniziative come il progetto HYDRAS (*HYdrodynamic-aware Distributed Robots for mArine Source-seeking*), volto a sviluppare sistemi multi-agente distribuiti in grado di localizzare sorgenti inquinanti in ambienti complessi attraverso l'integrazione di robotica cooperativa, apprendimento automatico e modellazione fisica delle correnti.

Studi precedenti sul rilevamento di pennacchi chimici in ambienti terrestri hanno prodotto ottimi risultati nella localizzazione di sorgenti inquinanti tramite il dispiegamento di squadre di robot mobili [1, 2]. L'utilizzo di sistemi multi-agente risulta dunque preferibile ad alternative basate sul tele-rilevamento satellitare, le quali offrono una risoluzione spaziale e temporale limitata e sono frequentemente compromesse da occlusioni ambientali come la copertura nuvolosa. Per quanto riguarda l'ambiente marino, i pennacchi

chimici vengono rilevati e monitorati attraverso il dispiegamento di veicoli autonomi sottomarini (AUV) [3] o imbarcazioni autonome di superficie (ASV) [4].

Il presente elaborato di tesi si inserisce in questo filone di ricerca sulla robotica marina autonoma, con l'obiettivo di definire algoritmi e strategie di coordinamento decentralizzate per flotte di agenti autonomi in ambienti confinati. In particolare, il sistema proposto si posiziona in una fase preliminare o complementare rispetto ai framework avanzati previsti da HYDRAS: lo scopo è realizzare un monitoraggio esplorativo volto a identificare le zone critiche e le sorgenti inquinanti prima del dispiegamento delle risorse di più alto livello. I dati raccolti da questa prima fase di ricognizione saranno utilizzati per lo sviluppo e l'addestramento di algoritmi intelligenti.

Il presente elaborato adotta un approccio geometrico decentralizzato basato su regole di coordinamento e filtraggio spaziale, volto a garantire un'elevata copertura dell'ambiente. Nello specifico: il Capitolo 2 descrive l'architettura logica della simulazione e le scelte di approssimazione introdotte per modellare il mondo reale, il Capitolo 3 tratta le soluzioni implementative adottate per lo sviluppo di un flusso operativo efficiente, in grado di simulare dinamicamente nel tempo l'azione cooperativa degli agenti marini sulla mappa in esame per l'ottenimento dei risultati statistici, il Capitolo 4 analizza in maniera critica i risultati ottenuti e il Capitolo 5 analizza i limiti della struttura logica della simulazione e accenna ad eventuali sviluppi futuri.

Capitolo 2

Mappa, Agenti e Meccanismi di Coordinamento

2.1 Mappa e griglia di copertura

Il progetto opera utilizzando una mappa reale dell'area costiera di Cecina, importata tramite il file di configurazione `sea_land_mask_10m_Cecina.npz`. Lo spazio geografico continuo è stato discretizzato mediante una griglia bidimensionale uniforme di dimensioni pari a 300×250 celle (pixel). All'interno del file, le maschere booleane identificano la distinzione tra celle di terra e di mare, dove ogni singolo elemento rappresenta un'area quadrata reale di 100 metri quadrati.

All'interno dell'area descritta sarà dispiegata una squadra di agenti mobili che navigheranno all'interno della mappa. Durante il movimento, ogni singola cella marina (identificata dalle coordinate discrete i e j) assume un valore di copertura $a_{ij}(t) \in [0.0, 1.0]$ che simula il grado di informazione locale che possediamo sulla cella medesima. Tale valore raggiunge il massimo nell'istante di tempo in cui la cella rientra nel raggio operativo di un agente. Per

simulare la successiva perdita di attualità temporale dell'informazione causata da correnti e da fenomeni di dispersione fisica tipici dell'ambiente marino, lo stato di copertura della singola cella subisce un processo di decadimento temporale costante, imponendo alla flotta la necessità di un pattugliamento continuo.

Di conseguenza, l'obiettivo principale del sistema è la massimizzazione del livello di informazione globale di questo specchio d'acqua, quantificato matematicamente attraverso il parametro dinamico di *Coverage Value*:

$$Coverage\ Value(t) = \sum_{(i,j) \in \Omega_{sea}} a_{ij}(t) \quad (2.1)$$

Tale valore rappresenta il grado di informazione totale e aggiornato di cui si dispone all'istante temporale t , ed è calcolato semplicemente come la somma dei valori di copertura di tutte le celle marine appartenenti al dominio di monitoraggio Ω_{sea} .

2.2 Agenti e algoritmo di pianificazione

Ciascun agente della flotta, definito fisicamente dal raggio del proprio sensore e dalla sua velocità di navigazione, è programmato per stabilire autonomamente la propria traiettoria, operando sulla base della conoscenza istantanea della posizione degli altri membri della flotta. Per determinare la futura destinazione, l'agente esegue un campionamento stocastico dell'area circostante secondo una distribuzione normale, la cui deviazione standard è direttamente parametrata in funzione del raggio del sensore installato a bordo. I punti candidati così ottenuti vengono sottoposti a un processo di filtraggio geometrico sequenziale volto a eliminare:

1. i punti che risultano spazialmente esterni al cerchio delimitato dal raggio del sensore dell'agente;
2. i punti candidati che risultano spazialmente più vicini a un altro agente rispetto all'agente stesso;
3. i punti candidati le cui traiettorie di viaggio intersecano ostacoli terrestri. Questo controllo viene eseguito tramite l'algoritmo di Bresenham [5] (la cui trattazione matematica è approfondita nella Sottosezione 3.5), che traccia una linea discreta tra l'agente e il target candidato, verificando l'assenza di celle di terra lungo il percorso.

Una volta filtrato l'insieme delle posizioni candidate, l'agente esegue una valutazione euristica: calcola il valore di copertura teorico che la mappa assumerebbe qualora decidesse di muoversi su ciascuna specifica coordinata, e seleziona infine la destinazione che restituisce il miglior incremento teorico di copertura complessiva (*Coverage Value*).

Il sistema è reso scalabile e resiliente grazie ad uno scambio informativo tra i robot ridotto al minimo indispensabile: gli agenti condividono esclusivamente le proprie posizioni istantanee reciproche, rendendo tale informazione sufficiente a calcolare i filtri di prossimità spaziale e a limitare conflitti durante la navigazione.

2.2.1 Architettura dell'agente

Gli agenti della flotta sono stati progettati come una macchina a stati finiti. Dal momento dell'attivazione fino allo spegnimento del sistema, ogni robot oscilla ciclicamente tra due stati operativi:

- **IDLE**: l'agente ha raggiunto le coordinate della destinazione precedentemente calcolata. In questo stato, il robot arresta il moto ed esegue

l'algoritmo di campionamento e filtraggio per calcolare il nuovo target, ed esegue il primo spostamento.

- **MOVING:** l'agente si sta muovendo attivamente verso il proprio target, occupandosi esclusivamente di seguire passo dopo passo la traiettoria lineare tracciata dall'algoritmo di Bresenham.

2.3 Semplificazioni del modello

Al fine di garantire l'efficienza computazionale del simulatore, sono state introdotte alcune approssimazioni cinematiche e modellistiche controllate:

- **Agenti puntiformi:** la dimensione fisica dei singoli robot è considerata trascurabile rispetto alla risoluzione geometrica della cella della mappa.
- **Isotropia della griglia:** la distanza tra il centro di una cella e quello delle celle adiacenti è considerata uniformemente pari a 10 metri, semplificando così il calcolo delle distanze spaziali ed evitando la discretizzazione metrica differenziata per i movimenti in diagonale.
- **Astrazione della cinematica dei fluidi:** l'effetto di deriva causato da correnti d'acqua e vento non è modellato cinematicamente sulle traiettorie dei robot, ma è trascurato, fornendo una navigazione lineare e priva di perturbazioni degli agenti.
- **Traiettorie lineari prive di deviazione:** le traiettorie degli agenti seguono segmenti retti continui tra il punto di partenza e il punto di arrivo. Non essendo previsti algoritmi di navigazione complessi per aggirare gli ostacoli in tempo reale o per evitare le collisioni tra

agenti, i target la cui traiettoria interseca un lembo di terra vengono preventivamente scartati in fase di filtraggio.

- **Tempo di calcolo target trascurabile:** gli agenti non stanno mai fermi, si presuppone che il tempo di calcolo della nuova posizione sia trascurabile.

2.4 Struttura della simulazione

2.4.1 Parametrizzazione

L'intero framework è altamente configurabile tramite un file ausiliario di costanti (`constants.py`). Attraverso questo modulo è possibile modificare centralmente l'intero scenario sperimentale, agendo su quattro categorie principali di parametri.

- **Parametri fisici e sensoriali dei droni:** la velocità dell'agente (`v_agent`), il raggio d'azione del sensore (`SENSOR_RANGE`), il numero di campioni stocastici generati per ciclo (`NUM_SAMPLES`) e il numero complessivo di droni della flotta (`NUM_AGENTS`).
- **Parametri ambientali:** la costante di decadimento di riferimento (`DECAY_RATE_REF`) e l'intervallo temporale nominale di applicazione (`TIMESTEP`).
- **Struttura della simulazione:** il passo temporale (`t`), il numero totale di iterazioni per singolo run (`NUM_ITERATIONS`) e il numero complessivo di run indipendenti per l'analisi statistica (`NUM_RUNS`).
- **Configurazione iniziale della flotta:** la modalità di *spawn* iniziale (`RANDOM_SPAWN`), consente di scegliere tra una partenza concentrata

da un unico porto fisso e un posizionamento casuale distribuito uniformemente all'interno di un'area circolare definita (`SPAWN_RADIUS`, `START_ROW`, `START_COL`).

2.4.2 Ciclo operativo

Il simulatore esegue l'avanzamento temporale attraverso istanti discreti. Ciascun passo temporale (t) rappresenta l'intervallo temporale discreto necessario a un agente, procedendo alla velocità nominale costante v_{agent} , per compiere lo spostamento lineare corrispondente alla larghezza di una cella della griglia (`pixel_dimension`), costante definita nel file `constants.py` la quale rappresenta una proprietà geometrica intrinseca dettata dal dataset cartografico della mappa in uso:

$$t = \frac{\text{pixel_dimension}}{v_{\text{agent}}} \quad (2.2)$$

All'avvio della simulazione, la dislocazione iniziale dei droni nello spazio può essere configurata secondo le due diverse modalità di cui sopra.

Il ciclo di esecuzione prevede l'inizializzazione della mappa e della flotta dei robot, seguita da un loop temporale in cui ad ogni iterazione viene calcolato l'aggiornamento sia dello stato che della posizione degli agenti, i quali scelgono la nuova destinazione a seconda dello stato operativo (*IDLE* o *MOVING*). Al termine di ogni iterazione, la mappa aggiorna lo stato delle celle applicando il tasso di decadimento temporale.

Per garantire la coerenza fisica temporale del decadimento, all'interno del file `constants.py` è possibile definire il valore di decadimento di riferimento (`DECAY_RATE_REF`) e l'intervallo di tempo nel quale quest'ultimo viene applicato (`TIMESTEP`). Una volta definite queste costanti, è necessario che il decadimento venga applicato ad ogni istante di tempo seguendo i dettami

della simulazione; per questo motivo, è necessario calcolare un nuovo valore di decadimento (`DECAY_RATE`) proporzionale alla durata del passo t .

Inizialmente si definisce la perdita informativa sulla base del valore di decadimento originale:

$$\text{Loss}_{\text{ref}} = 1 - \text{DECAY_RATE_REF} \quad (2.3)$$

Tale valore viene riscalato linearmente, al fine di ottenere la reale perdita applicabile ad ogni iterazione, nel modo seguente:

$$\text{Loss} = \text{Loss}_{\text{ref}} \times \frac{t}{\text{TIMESTEP}} \quad (2.4)$$

Il tasso di decadimento effettivo (`DECAY_RATE`) risulta quindi pari a:

$$\text{DECAY_RATE} = 1 - \text{Loss} \quad (2.5)$$

Al termine della sessione di simulazione, i dati grezzi raccolti vengono salvati su disco tramite file di log serializzati. Questi file costituiscono l'input per il successivo script di analisi dedicato, il quale si occupa di rigenerare tabelle prestazionali, grafici di andamento temporale e animazioni video dei run simulati.

Capitolo 3

Implementazione

3.1 Tecnologie e ambiente di sviluppo

L'intero simulatore software è stato sviluppato utilizzando il linguaggio di programmazione **Python** (versione 3.12), scelto per l'esteso ecosistema di moduli dedicati al calcolo scientifico e ingegneristico.

Il funzionamento del simulatore e la successiva fase di analisi dei dati si appoggiano esclusivamente ad alcune librerie scientifiche e grafiche fondamentali:

- **NumPy (v2.5.1)**: gestisce il calcolo scientifico multidimensionale, la manipolazione delle maschere booleane e le operazioni matematiche di decadimento della griglia.
- **Matplotlib (v3.11.0)**: genera i grafici di andamento della copertura, gli istogrammi statistici e la visualizzazione bidimensionale dei singoli istanti di simulazione.
- **Imageio (v2.37.3) con modulo Imageio-FFmpeg (v0.6.0)**: consente la compilazione sequenziale automatica dei singoli frame PNG

salvati in un filmato in formato *MP4*.

- **Pillow (v12.3.0):** fornisce il supporto fondamentale per la manipolazione, il salvataggio e l'elaborazione dei singoli frame grafici generati dal simulatore.

Le simulazioni e le successive elaborazioni dei dati sono state eseguite su un elaboratore avente le seguenti specifiche di sistema:

- **CPU:** Intel Core i7-13620H (2.40 GHz).
- **RAM:** 16 GB (4800 MT/s).
- **OS:** Windows 11 (64-bit).

3.2 Architettura software

Il software sviluppato si presenta organizzato secondo una struttura modulare descritta dal seguente albero delle directory:

```
multi-agent-monitoring
|-- .venv
|-- core
|   |-- agent.py
|   |-- bresenham_utilities.py
|   |-- map.py
|   |-- spawn_utilities.py
|   |--plots_multi_run
|-- results
|   |-- data
|   |-- plots
```

```
|    |-- video
|-- visualization
|--|-- compare_runs.py
|    |-- plots.py
|    |-- renderer.py
|-- .gitattributes
|-- .gitignore
|-- analyze.py
|-- analyze_multi_run.py
|-- constants.py
|-- main.py
|-- README.md
|-- requirements.txt
|-- sea_land_mask_10m_Cecina.npz
|-- sea_land_mask_10m_Cecina.png
```

La directory `core` costituisce il nucleo computazionale principale dell'elaborato, all'interno della quale sono definiti i metodi che caratterizzano lo stato e le decisioni di movimento degli agenti, nonché i comportamenti e le trasformazioni applicabili alla griglia della mappa. I file contrassegnati come `_utilities` contengono invece i metodi geometrici e statistici ausiliari necessari al corretto supporto dello scenario simulativo.

Lo script principale `main.py` coordina l'esecuzione del simulatore richiamando le classi e le funzioni definite all'interno della directory `core`. Questo processo genera i log dei dati grezzi che vengono successivamente memorizzati all'interno della cartella `results/data` per poter essere analizzati asincronamente.

La directory `visualization` raccoglie i moduli software deputati al trattamento e alla rappresentazione dei dati grezzi prodotti durante l'esecuzione del simulatore. Questi componenti elaborano i log per produrre gli output visivi di interesse, memorizzandoli ordinatamente nelle rispettive sottocartelle `results/plots` (per grafici e tabelle di copertura) e `results/video` (per i filmati dinamici della navigazione); parallelamente, la cartella `plots_multi_run` raccoglie gli elaborati visivi derivanti dal confronto tra simulazioni condotte con differenti configurazioni parametriche.

Infine, in modo complementare allo script di simulazione, gli script `analyze.py` e `analyze_multi_run.py` si occupano di gestire e orchestrare l'intero flusso di lavoro finalizzato all'elaborazione automatica e alla generazione finale dei report grafici.

3.3 Rappresentazione della mappa

La mappa viene caricata contestualmente all'istanziamento dell'oggetto `Map`; quest'ultimo riceve in ingresso il nome del file e nel suo costruttore si occupa di estrarne i dati utili ai fini del progetto. Nello specifico, vengono estratti gli array `x` e `y`, contenenti i riferimenti spaziali della griglia, e le matrici booleane `sea_mask` e `land_mask`, utilizzate rispettivamente per identificare le celle navigabili (corrispondenti a 39.330 celle di mare) e i limiti geometrici della terraferma. Sulla base di questi dati, il costruttore definisce la dimensione complessiva del dominio (`shape`) e inizializza la matrice dinamica dei valori di monitoraggio (`grid`) con valori nulli, predisponendo il sistema alla gestione e al calcolo in tempo reale dell'informazione accumulata (`coverage_value`). Il file `map.py` contiene i metodi descrittivi delle azioni eseguibili sulla mappa. Nello specifico, all'interno della classe `Map` vengono

implementate le seguenti funzioni fondamentali per la gestione dello scenario simulativo e per il calcolo delle metriche di copertura:

- `query_theoretical_coverage(radius, row, col)`: calcola in via preventiva il valore di copertura globale che la flotta otterrebbe qualora un agente decidesse di spostarsi sulla cella marina di coordinate `(row, col)`. Il metodo interroga l'algoritmo geometrico per identificare tutte le celle marine visibili da quella coordinata e ne calcola il potenziale incremento di informazione, quantificato per ogni cella come la differenza $1.0 - \text{grid}[r, c]$;
- `update_coverage_value()`: aggiorna il parametro complessivo di informazione accumulata della flotta (`coverage_value`), calcolando la somma dei valori correnti di copertura presenti nella griglia limitatamente alle sole celle appartenenti alla maschera di mare (`sea_mask`);
- `is_sea(row, col)`: esegue un controllo di sicurezza preliminare per evitare errori di indicizzazione (*IndexError*), restituendo un valore booleano che attesta se la coordinata specificata si trovi all'interno dei limiti geometrici della griglia e corrisponda effettivamente a una cella marina navigabile;
- `cell_view(row, col, radius)`: simula l'azione di scansione sensoriale di un agente sulla coordinata `(row, col)`. Il metodo identifica le celle visibili entro il raggio d'azione del sensore del robot, ne imposta il valore di copertura a 1.0 (informazione massima aggiornata) e provvede ad aggiornare il punteggio di copertura globale della mappa;
- `decay(decay_rate)`: applica la degradazione temporale dell'informazione su tutta l'area marina del dominio. I valori di copertura correnti vengono moltiplicati vettorialmente per il tasso di decadimento

(`decay_rate (2.5)`), limitando i valori risultanti nell'intervallo $[0.0, 1.0]$ tramite un'operazione di *clipping* per scongiurare errori numerici di arrotondamento, aggiornando infine lo stato globale.

3.4 Logica dell'agente

Gli agenti vengono istanziati durante il workflow principale, ricevendo i dati necessari e il riferimento alla mappa condivisa utilizzata per coordinare le rispettive posizioni con gli altri membri della flotta; inoltre, nel costruttore viene subito registrata la presenza iniziale dell'agente sulla mappa tramite il metodo `cell_view` descritto in precedenza. Nello specifico, il costruttore riceve in ingresso le coordinate di riga (`start_row`) e colonna (`start_col`) di avvio (di cui viene preventivamente verificata la navigabilità), il raggio d'azione del sensore espresso in celle (`sensor_range`) e il numero di candidati stocastici da generare ad ogni ciclo (`num_samples`).

All'interno della classe **Agent** sono implementati i metodi descrittivi delle logiche decisionali e di movimento che governano il comportamento dinamico del veicolo:

- `get_samples()`: genera un insieme di coordinate candidate per lo spostamento futuro dell'agente. I campioni vengono estratti secondo una distribuzione normale bidimensionale centrata sulla posizione corrente dell'agente e con deviazione standard proporzionale al raggio del sensore; il metodo esclude preventivamente i punti situati sulla terraferma o aventi una distanza euclidea superiore al raggio d'azione sensoriale;
- `filter_samples(samples, other_agents)`: filtra l'insieme dei campioni stocastici applicando due vincoli geometrico-operativi. Il primo esclude i punti che risultano spazialmente più vicini a un altro robot

rispetto all'agente stesso; il secondo sfrutta la funzione `is_path_free` definita in `bresenham_utilities.py` (Sottosezione 3.5) per scartare i punti la cui traiettoria rettilinea di navigazione interseca ostacoli terrestri;

- `find_goal_point(samples)`: esegue una valutazione euristica greedy sui campioni che hanno superato la fase di filtraggio. Il metodo interroga la funzione `query_theoretical_coverage` della mappa per determinare quale target garantisca il massimo incremento di informazione globale, restituendone le coordinate riga e colonna;
- `move_to(row, col)`: aggiorna la posizione fisica dell'agente impostando le nuove coordinate spaziali e provvede ad aggiornare i valori di copertura della griglia visibile dal nuovo punto richiamando il metodo `cell_view` della mappa;
- `find_next_position(other_agents)`: governa il comportamento logico della macchina a stati finiti (FSM Figura 3.1). Se lo stato è impostato su `MOVING`, estrae e restituisce il passo successivo lungo il percorso pre-calcolato (`self.path`). Se si trova in stato `IDLE`, attiva la sequenza di ricerca del nuovo obiettivo (campionamento, filtraggio e selezione target), calcola la nuova traiettoria tramite l'algoritmo di Bresenham, commuta lo stato su `MOVING` ed esegue immediatamente il primo passo del cammino;
- `update_position(other_agents)`: coordina l'aggiornamento complessivo dell'agente ad ogni iterazione temporale. Per determinare la cella adiacente di arrivo richiama `find_next_position`, sposta fisicamente l'agente tramite `move_to` e restituisce alla simulazione il valore aggiornato di copertura globale calcolato dalla mappa.

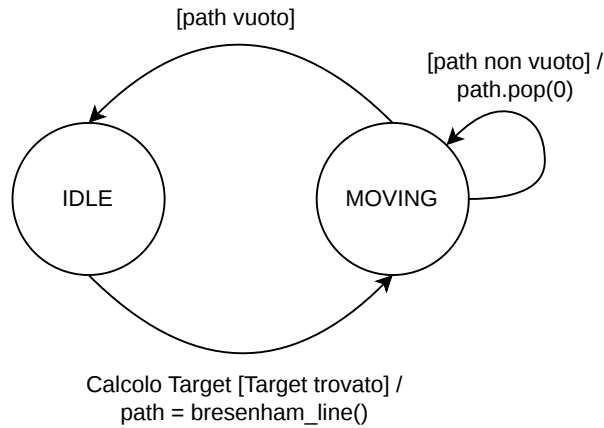


Figura 3.1: Diagramma degli stati dell'agente (Modello di Mealy).

In un'ottica di sviluppi futuri e per agevolare il testing comparativo di differenti algoritmi di navigazione, è possibile ipotizzare un'ulteriore modularizzazione dell'architettura software attraverso il disaccoppiamento della logica decisionale dell'agente. Nello specifico, si potrebbe adottare un *Design Pattern* di tipo *Strategy* (o *Controller*), in cui la classe **Agent** mantiene un riferimento a un oggetto controllore esterno, delegando a quest'ultimo il calcolo del target e della traiettoria. Così facendo, l'iniezione di controllori differenti consentirebbe di testare e confrontare algoritmi diversi sulla medesima flotta senza dover attuare riscritture strutturali o modifiche drastiche alla classe principale. Nel presente elaborato, al fine di preservare l'immediata leggibilità del codice e garantire la semplicità strutturale del simulatore, si è scelto di mantenere la logica algoritmica integrata direttamente all'interno della classe **Agent**.

3.5 Algoritmi di supporto

Per ottimizzare la struttura del simulatore e garantire l'assoluta modularità del codice, sono stati implementati moduli di utilità esterni alle classi principali **Map** e **Agent**. Questi moduli contengono algoritmi geometrici e di campionamento statistico fondamentali per la gestione dello scenario di simulazione.

3.5.1 Modulo `bresenham_utilities.py`

Questo file raccoglie la definizione e le applicazioni dell'algoritmo di Bresenham, un metodo di discretizzazione geometrica che permette di approssimare segmenti rettilinei continui su griglie bidimensionali bitmap. All'interno del modulo vengono definiti tre metodi principali:

- `bresenham_line(r0, c0, r1, c1)`: la classica implementazione dell'algoritmo incrementale di Bresenham per il tracciamento di linee su griglia. L'algoritmo opera esclusivamente con aritmetica intera, utilizzando solo somme e sottrazioni, evitando costose operazioni in virgola mobile o divisioni. Definito l'errore decisionale iniziale basato sulle differenze spaziali delle coordinate di partenza e di arrivo ($\Delta r = |r_1 - r_0|$ e $\Delta c = |c_1 - c_0|$), l'algoritmo accumula l'errore passo dopo passo lungo l'asse principale. Ad ogni incremento, determina in modo deterministico quale cella adiacente minimizzi lo scostamento rispetto alla retta continua ideale, restituendo la sequenza ordinata di coordinate discrete (r, c) che uniscono i punti di partenza e di arrivo.
- `is_path_free(r0, c0, r1, c1, map_reference)`: utilizza il metodo `bresenham_line` per tracciare la traiettoria rettilinea tra due punti e verificare la totale assenza di ostacoli terrestri lungo il percorso. Se

anche una sola cella della sequenza discreta appartiene alla maschera di terraferma (`land_mask`), la funzione restituisce `False`, segnalando che la rotta è ostruita. Un'attenzione particolare è stata rivolta alla gestione dei passi diagonali: a causa della discretizzazione su griglia a 8-connettività, un movimento diagonale potrebbe consentire al drone di attraversare impropriamente un lembo sottile di terraferma qualora la linea congiunga due celle di mare contigue per vertice, ma separate da scogli o moli diagonalmente adiacenti (Figura 3.2). Per prevenire tale fenomeno, ogni volta che uno spostamento elementare comporta una variazione contemporanea su entrambi gli assi (x e y), la funzione esegue un controllo rigoroso sulle due celle ortogonali adiacenti: qualora anche una sola di esse appartenga alla terraferma, la traiettoria viene considerata non navigabile e preventivamente scartata. Questo controllo geometrico risulta cruciale nella fase di filtraggio dei campioni stocastici dell'agente (`filter_samples`), descritta nella Sottosezione 3.4.

- `get_visible_cells(r0, c0, radius, map_reference)`: implementa una tecnica di *ray-casting* per determinare l'insieme delle celle marine visibili dall'agente entro il proprio raggio sensoriale. Per ciascuna cella situata entro la distanza euclidea stabilita dal raggio (`radius`), la funzione verifica la linea di vista a partire dal centro dell'agente (r_0, c_0) avvalendosi direttamente del metodo `is_path_free`. Se il segmento incontra una cella di terraferma, il raggio visivo viene interrotto, impedendo all'agente di rilevare (o aggiornare nel simulatore) le celle marine poste dietro ostacoli fisici terrestri (come penisole o isole).

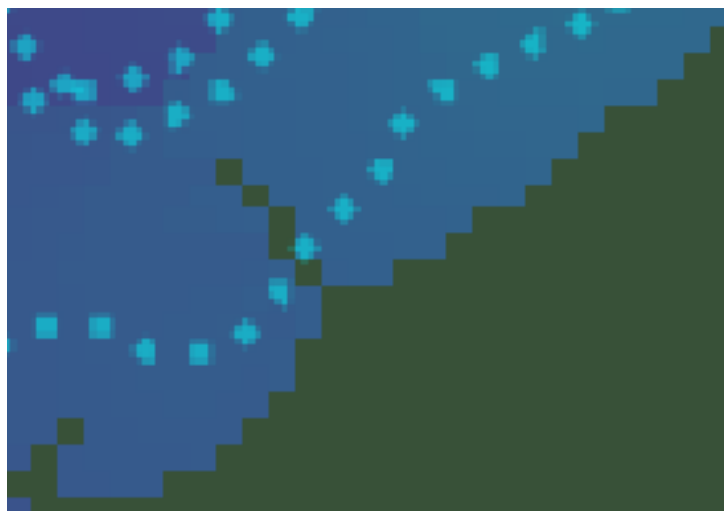


Figura 3.2: Caso limite di attraversamento terraferma.

L'algoritmo di Bresenham traduce dunque una linea retta continua in una sequenza discreta di pixel, alternando passi ortogonali e passi diagonali. Da un punto di vista geometrico, poiché lo spostamento diagonale su una griglia quadrata copre una distanza maggiore di un fattore $\sqrt{2}$ rispetto a quello ortogonale nel medesimo passo temporale discreto, una traiettoria puramente diagonale a 45° comporterebbe un incremento massimo di velocità teorica di circa il 41.4% rispetto al valore nominale. Tuttavia, poiché le rotte generate dai campionamenti si sviluppano lungo angolazioni arbitrarie a 360° , la sequenza di Bresenham risultante riduce notevolmente lo scostamento cinematico medio effettivo, rendendo tale approssimazione ampiamente accettabile ai fini dell'esplorazione della mappa.

3.5.2 Modulo `spawn_utilities.py`

Questo file ospita un unico metodo adibito all'inizializzazione spaziale della flotta all'avvio del programma:

- `random_spawn(start_row, start_col, spawn_radius, num_agents, map_reference)`: definisce le posizioni di partenza degli agenti all'avvio della simulazione. Ricevute in ingresso le coordinate centrali dell'area di spawn, la funzione isola tutte le celle marittime navigabili situate entro la distanza euclidea definita dal raggio di spawn (`spawn_radius`). Da questo sottoinsieme di coordinate valide, il metodo estrae casualmente un numero di posizioni pari a `num_agents` seguendo una distribuzione uniforme senza ripetizione, sollevando un'eccezione qualora il numero di celle marine idonee risulti insufficiente per ospitare l'intera flotta.

3.6 Gestione del ciclo di simulazione

Il flusso di esecuzione principale del simulatore è interamente coordinato all'interno del file `main.py`, il quale si occupa dell'inizializzazione dello scenario, dell'avanzamento temporale e dell'aggregazione dei dati necessari per ottenere le rappresentazioni grafiche e statistiche desiderate.

- **Inizializzazione dello Scenario e Configurazione dello Spawn:**
All'inizio di ciascuna delle run indipendenti (`NUM_RUNS`), il simulatore provvede a inizializzare l'ambiente istanziando la classe della mappa condivisa (`Map`) e assegnando la posizione di partenza a ciascun agente secondo le modalità prestabilite. Successivamente, vengono creati i singoli elementi della flotta (`Agent`), iniettando in ciascun costruttore il riferimento alla mappa comune e le costanti operative definite nel file `constants.py`.
- **Il Loop Temporale e l'Aggiornamento della Griglia:**
Una volta predisposto lo scenario, ha inizio il ciclo temporale discreto

composto da un numero programmato di iterazioni (`NUM_ITERATIONS`). Ad ogni passo temporale, il loop di simulazione prevede le seguenti operazioni sequenziali:

1. Invoca per ciascun agente il metodo `update_position(fleet)`, il quale si occupa di richiamare la sequenza di funzioni atte a consentire il processo di decisione e di movimento discreto sulla griglia;
2. Applica la degradazione dei valori di copertura su tutta la griglia marina richiamando la funzione di decadimento della mappa.

- **Acquisizione dei Log e Ottimizzazione della Memoria:**

Durante l'avanzamento del ciclo temporale, lo script acquisisce in tempo reale i dati operativi di interesse. In particolare, per il primo run di simulazione, vengono registrati dettagliatamente i vettori delle traiettorie spaziali istantanee degli agenti e l'andamento del livello di copertura globale. Al contempo, per l'insieme di tutte le run eseguite, vengono collezionati i dati storici grezzi del valore di copertura.

Al fine di ottimizzare l'occupazione di memoria e di ridurre il volume dei dati salvati nei file di log, le matrici bidimensionali della mappa non vengono memorizzate per ogni singolo run, bensì viene calcolata e salvata direttamente la matrice dello stato della griglia mediata su tutte le `NUM_RUNS` esecuzioni a intervalli regolari (`ITERATIONS_STEP`).

3.7 Post-processing dei dati

Al fine di garantire l'efficienza computazionale della simulazione ed evitare sovraccarichi durante i calcoli numerici, il sistema adotta un'architettura

software disaccoppiata. La generazione degli output statistici, delle tabelle, dei grafici di confronto e del rendering video non avviene all'interno del ciclo principale di calcolo, bensì viene gestita in differita dagli script di post-processing, che operano rileggendo i log binari precedentemente salvati su disco. Inoltre, per facilitare l'interpretazione dei risultati grafici e numerici, le grandezze monitorate sono state parametrizzate in funzione del reale tempo fisico trascorso (ore, minuti e secondi), anziché del conteggio delle iterazioni discrete.

3.7.1 Elaborazione statistica dei dati

Gli script di analisi accedono ai log serializzati (`media_data`) in modalità di lettura binaria ("`rb`"), gestendo in modo robusto le eventuali eccezioni di assenza dei file (`FileNotFoundError`). Una volta caricati in memoria i dizionari tramite la libreria `pickle`, i dati numerici vengono sottoposti al medesimo trattamento statistico: i vettori storici dei valori di copertura di ciascuna run (con $r \in \{1, \dots, \text{NUM_RUNS}\}$) vengono aggregati per calcolarne il valore medio assoluto $\mu(t)$, la corrispondente percentuale media e la relativa deviazione standard ad ogni istante temporale t .

Definito $c_r(t)$ come il valore di copertura assoluto ottenuto al run r -esimo, la media aritmetica (`np.mean`) è espressa da:

$$\mu(t) = \frac{1}{\text{NUM_RUNS}} \sum_{r=1}^{\text{NUM_RUNS}} c_r(t) \quad (3.1)$$

Tale valore medio assoluto viene successivamente rapportato al numero totale delle celle marine navigabili (`NUM_SEA_CELLS`, corrispondente al valore di copertura massimo teorico qualora tutte le celle presentassero un valore di informazione pari a 1.0 nello stesso istante) per ricavare la copertura percentuale media.

La deviazione standard percentuale $\sigma_{\%}(t)$, calcolata tramite l'operatore statistico `np.std` per quantificare lo scostamento medio delle singole esecuzioni ed esprimere la stabilità statistica complessiva dell'algoritmo, è definita come:

$$\sigma_{\%}(t) = \left(\sqrt{\frac{1}{\text{NUM_RUNS}} \sum_{r=1}^{\text{NUM_RUNS}} (c_r(t) - \mu(t))^2} \right) \times \frac{100}{\text{NUM_SEA_CELLS}} \quad (3.2)$$

Questi calcoli statistici permettono di ripulire i risultati finali dalle fluttuazioni causate dal campionamento stocastico. I dati così elaborati vengono quindi passati alle funzioni ausiliarie per la generazione delle rispettive rappresentazioni grafiche.

3.7.2 Analisi del singolo scenario

Nello script `analyze.py`, il flusso esecutivo si divide in due fasi distinte: la prima dedicata alla statistica analitica e la seconda al rendering dinamico dello scenario.

Nella prima fase, attraverso le funzioni del modulo ausiliario della grafica (`plots.py`), il sistema processa i dati aggregati per esportare tre rappresentazioni analitiche fondamentali:

- **tabelle prestazionali:** campionano la cronologia dei dati ad intervalli regolari, mostrando il tempo di simulazione, la copertura media e la relativa deviazione standard del sistema;
- **grafici di andamento:** tracciano la curva temporale della copertura marina lungo l'intero arco delle iterazioni per valutare la velocità di convergenza della flotta;

- **istogrammi spaziali:** analizzano lo stato della griglia di mare ad intervalli regolari, per mostrare la distribuzione di frequenza dei valori di copertura, evidenziando quanta parte di territorio marino sia stata mantenuta a determinati livelli di aggiornamento informativo.

La seconda fase si occupa di ricostruire visivamente la simulazione frame-by-frame a partire dal log del primo run operativo (*first run*):

- **rappresentazione cartografica:** lo stato della mappa viene ricostruito applicando sequenzialmente il decadimento e il raggio sensoriale dei droni, colorando in modo differenziato la terraferma, il mare inesplorato e le celle marine visitate;
- **tracciamento dei vettori e dei sensori:** sul piano cartografico vengono disegnati l'anello dell'area di spawn, le linee delle traiettorie storiche percorse, l'area coperta istantaneamente dal sensore e le icone che indicano la posizione di ogni drone;
- **compilazione del filmato:** i singoli frame disegnati vengono salvati temporaneamente e convertiti in tempo reale dal modulo scrittore (`renderer.py`) in un video finale fluido in formato *MP4*.

Invece di salvare la matrice di copertura dell'intera griglia per ogni istante temporale, si è scelto di ricalcolare questi dati durante il rendering seguendo le traiettorie del primo run (*first run*) e verificando la congruenza con i valori di copertura salvati. Questa scelta consente di alleggerire notevolmente i file di log generati, reputando il costo computazionale di un ricalcolo asincrono di gran lunga inferiore rispetto al salvataggio e all'immagazzinamento di una grande quantità di dati su disco.

3.7.3 Analisi comparativa multi-run

In modo complementare, lo script `analyze_multi_run.py` si occupa di confrontare tra loro scenari sperimentali differenti ottenuti variando i parametri di configurazione. Lo script estrae i dati aggregati dai rispettivi file `media_data` e delega al modulo `compare_runs.py` la produzione di due rappresentazioni comparative:

- **grafici a curve sovrapposte:** tracciano sullo stesso piano cartesiano l'andamento temporale della copertura media dei diversi test. Grazie al riscaldamento dell'asse delle ascisse, ottenuto convertendo le iterazioni in minuti, è possibile effettuare un confronto omogeneo anche tra simulazioni condotte a velocità differenti (caratterizzate da un numero eterogeneo di iterazioni, da 360 a 1080 passi);
- **grafici di trend discreto:** estraggono l'ultimo valore della curva di copertura percentuale, corrispondente al raggiungimento dello stato di regime stazionario al termine della simulazione, e lo associano al parametro di riferimento (ad esempio il numero di agenti). Il grafico a punti discreti così generato evidenzia visivamente la variazione della copertura a regime al variare delle risorse impiegate.

Capitolo 4

Risultati sperimentali

In questo capitolo vengono presentati e analizzati criticamente i risultati ottenuti dalle simulazioni numeriche del sistema di monitoraggio multi-agente proposto. L'obiettivo delle prove sperimentali è verificare l'efficacia e la stabilità statistica dell'algoritmo geometrico decentralizzato nel mantenere aggiornato lo stato informativo della mappa, contrastando attivamente il fenomeno di decadimento temporale.

Per garantire il realismo fisico dello scenario, i test sono stati eseguiti configurando il simulatore con i parametri operativi reali definiti nel file `constants.py`. Di seguito vengono riassunte le specifiche fisiche, geometriche e temporali adottate per la campagna di validazione.

- **Dominio Geografico (Cecina):** griglia discreta caricata dal file geografico `FILE_NAME`, caratterizzata da una risoluzione spaziale di $10\text{ m} \times 10\text{ m}$ per cella (`pixel_dimension`) e contenente esattamente 39.330 celle marine navigabili (`NUM_SEA_CELLS`, corrispondenti a circa 3.93 km^2 di superficie totale).
- **Consistenza della Flotta:** 10 droni marini autonomi cooperativi

(NUM_AGENTS) che operano in modo concorrente e decentralizzato all'interno dello scenario.

- **Specifiche e Sensori dei Droni:** velocità nominale costante di 2 m/s (`v_agent`), sensore di bordo con raggio d'azione pari a 8 celle (equivalenti a 80 m reali, `SENSOR_RANGE`) e capacità di pianificazione locale impostata a 50 candidati stocastici estratti per ciclo (`NUM_SAMPLES`).
- **Parametri di Decadimento:** tasso nominale di riferimento pari a 0.95 (`DECAY_RATE_REF`) applicato su un intervallo temporale di 30 s (`TIMESTEP`), successivamente riscalato dal simulatore per calcolare la perdita lineare associata ad ogni singola iterazione (vedi formula 2.5).
- **Discretizzazione Temporale:** passo temporale elementare t impostato a 5 secondi, calcolato fisicamente come rapporto tra la dimensione lineare della cella e la velocità del drone per garantire che ogni ciclo corrisponda allo spostamento spaziale esatto di una singola cella (vedi formula 2.2).
- **Metodologia di Test:** esecuzione di 100 run indipendenti (`NUM_RUNS`) per abbattere la deviazione standard statistica del campionamento stocastico ed estrarre un andamento medio pulito e affidabile.
- **Dislocazione Iniziale (Spawn):** posizionamento casuale abilitato (`RANDOM_SPAWN = True`) distribuito uniformemente entro un'area circolare di raggio $R_{\text{spawn}} = 20$ celle (200 m reali, variabile `SPAWN_RADIUS`) centrata sulle coordinate portuali nominali (100, 150) (`START_ROW`, `START_COL`).
- **Durata della Missione:** 720 iterazioni per singolo run (`NUM_ITERATIONS`), equivalenti a esattamente 1 ora di operazione continua reale considerando l'intervallo temporale di 5 secondi.

Durante la fase di validazione, i parametri operativi definiti nel modulo `constants.py` sono stati variati sistematicamente per condurre uno studio parametrico del sistema. Partendo dalla configurazione nominale di base appena descritta, le variabili di interesse (quali la consistenza numerica della flotta o la velocità dell'agente) sono state modificate singolarmente. Questo approccio metodologico ha permesso di isolare l'impatto di ciascuna grandezza e di valutare le prestazioni dell'algoritmo al mutare delle risorse a disposizione.

4.1 Analisi temporale del valore di copertura

I risultati quantitativi relativi all'andamento della copertura e della stabilità statistica della flotta, campionati a intervalli regolari di step temporali (`ITERATIONS_STEP`), sono raccolti nella tabella prestazionale di Figura 4.1.

Time (min)	Coverage Value	Coverage (%)	Std Dev (%)
0	969.94	2.47	0.29
5	6120.17	15.56	2.36
10	10312.24	26.22	3.41
15	12977.73	33.00	2.90
20	14778.79	37.58	2.21
25	15808.21	40.19	1.74
30	16238.77	41.29	1.54
35	16445.48	41.81	1.43
40	16579.28	42.15	1.43
45	16605.81	42.22	1.23
50	16612.49	42.24	1.08
55	16584.41	42.17	1.24
60	16532.14	42.03	1.19

Figura 4.1: Tabella dei valori di copertura.

All'istante iniziale ($0'$), la copertura percentuale media della mappa corrisponde unicamente alla quota minima associata all'impronta sensoriale iniziale della flotta concentrata all'interno dell'area di spawn circoscritta. Non appena ha inizio il ciclo dinamico, la percentuale di area coperta cresce in modo rapido e quasi lineare, man mano che gli agenti si allontanano dal porto ed esplorano le porzioni di mare libero adiacenti.

Come evidenziato dal grafico di andamento temporale di Figura 4.2, la curva di crescita della copertura presenta un andamento tipicamente asintotico. La pendenza della curva si riduce progressivamente nel tempo: questo fenomeno indica che, all'aumentare dello spazio scoperto, l'azione di acquisizione di informazione delle celle compiuta dai droni inizia ad essere contrastata in modo sempre più significativo e progressivo dall'azione di decadimento della mappa.

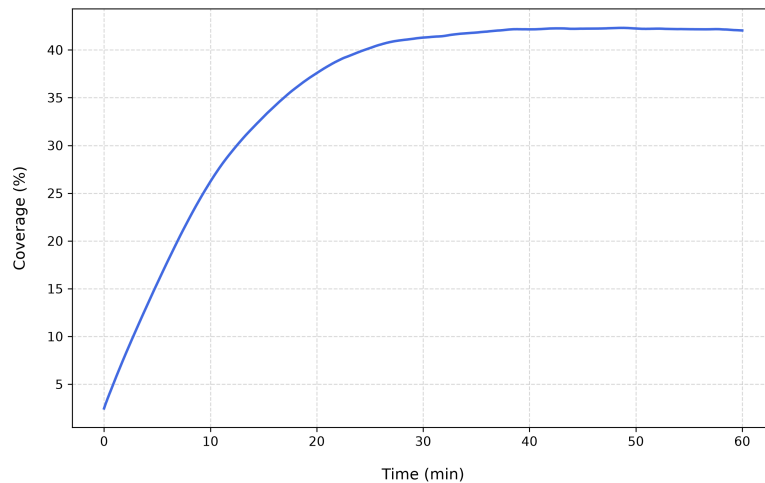


Figura 4.2: Andamento temporale del valore di copertura percentuale.

Il sistema raggiunge lo **stato stazionario (equilibrio dinamico)** intorno alla metà della simulazione, stabilizzandosi su un valore di copertura costante.

Un risultato di rilievo riguarda l'andamento della deviazione standard (vedi formula 3.2), visibile nei dati di Figura 4.1. Dopo un picco massimo registrato durante la fase di massima espansione geografica dei droni, la deviazione standard si riduce costantemente fino a stabilizzarsi su valori minimi all'istante finale. La progressiva riduzione della deviazione standard dimostra la convergenza e la stabilità statistica del controllo decentralizzato proposto: indipendentemente dallo spawn iniziale e dalla componente aleatoria del campionamento stocastico, la flotta si auto-organizza sempre nello stesso modo, garantendo la medesima efficienza di monitoraggio globale.

4.2 Distribuzione spaziale del valore di copertura

Al fine di comprendere la dinamica con cui la flotta distribuisce l'informazione sulla griglia e contrasta la perdita di dati, si è analizzato lo spostamento temporale della distribuzione di frequenza dei valori di copertura delle celle marine. La scelta di costruire istogrammi a partire dalla matrice media di tutte le run eseguite permette, come già anticipato in precedenza, di estrarre il valore medio della qualità del monitoraggio a lungo termine, eliminando la dipendenza stocastica dal campionamento normale delle prossime posizioni degli agenti e dal dispiegamento iniziale della flotta.

L'evoluzione spaziale della copertura viene analizzata in tre fasi cruciali della simulazione: lo stato transitorio iniziale (Figura 4.3), lo stato intermedio (Figura 4.4) e lo stato finale (Figura 4.5).

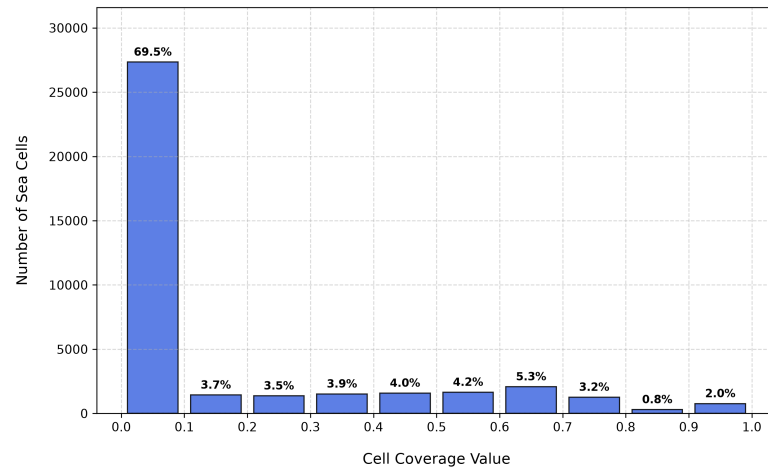


Figura 4.3: Distribuzione statistica del livello di copertura sulle celle di mare (5').

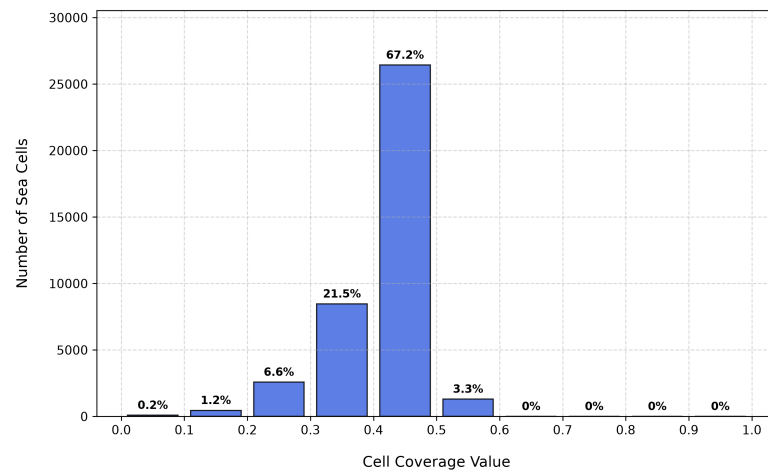


Figura 4.4: Distribuzione statistica del livello di copertura sulle celle di mare (30').

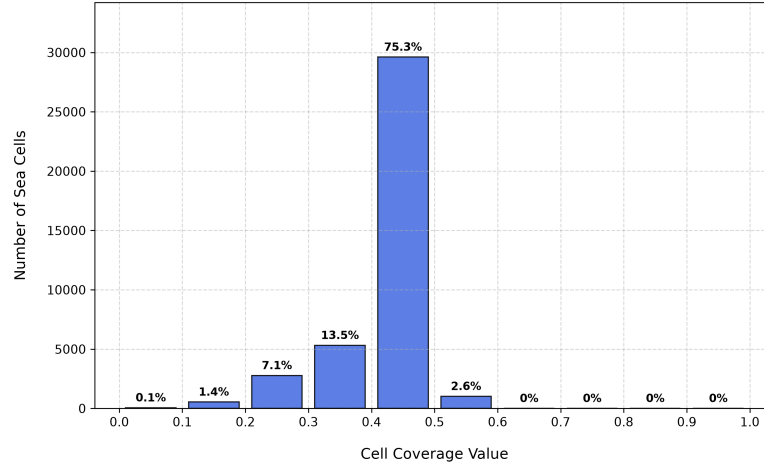


Figura 4.5: Distribuzione statistica del livello di copertura sulle celle di mare (60').

L'analisi comparativa dei tre istogrammi evidenzia una transizione di stato altamente regolare. Nella fase transitoria iniziale (5'), l'istogramma si presenta fortemente asimmetrico e sbilanciato verso sinistra; la maggior parte delle celle marine risiede nell'intervallo di copertura minimo, poiché la flotta è stata appena rilasciata e ampie porzioni del golfo di Cecina non sono ancora state raggiunte dagli agenti.

Procedendo nella simulazione, lo stato intermedio (30') coincide geometricamente con il punto che inizia a determinare il comportamento asintotico della curva di copertura, mostrando una drastica redistribuzione delle frequenze: la barra della copertura minima si azzerava quasi completamente, mentre la quasi totalità della griglia si sposta verso destra, addensandosi in una campana concentrata attorno ai valori intermedi di copertura.

A regime stazionario finale (60'), la distribuzione si consolida in maniera molto simile a quella registrata a metà simulazione. Questo comportamento dimostra l'emergere di un equilibrio dinamico in cui la grande maggioranza delle celle marine navigabili viene mantenuta costantemente all'interno di un range di aggiornamento intermedio. La quota irrilevante di celle rimaste

nei valori di minima copertura attesta l'efficacia dell'algoritmo di controllo adottato, impedendo ai robot di ammassarsi e garantendo un pattugliamento capillare evitando la formazione di zone d'ombra.

4.3 Analisi visiva e comportamento dinamico della flotta

La validazione qualitativa delle strategie di pianificazione locale e dei vincoli fisici di navigazione della flotta è stata condotta analizzando direttamente le mappe cartografiche generate dal simulatore durante i run operativi. Mentre i dati numerici precedentemente discussi attestano quantitativamente l'efficacia del sistema, l'analisi visiva delle mappe di Figure 4.6, 4.7 e 4.8 fornisce la dimostrazione pratica dei comportamenti fondamentali che governano la flotta.

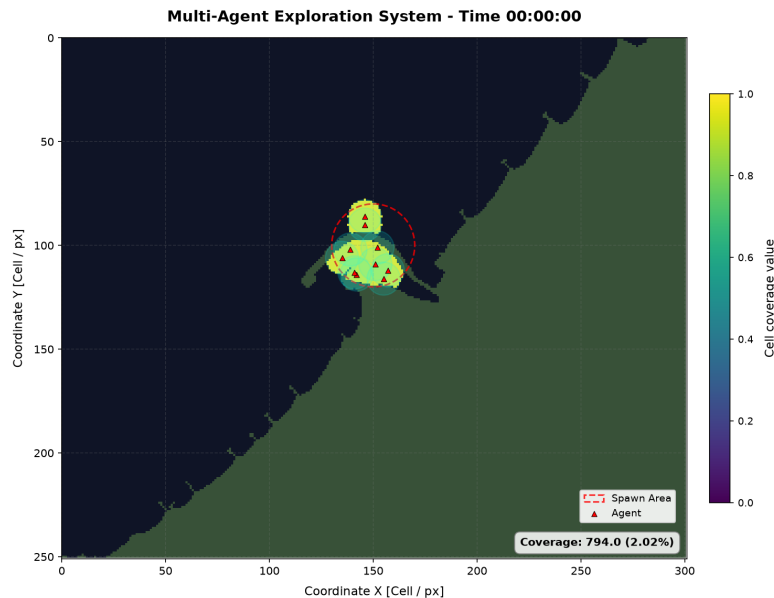


Figura 4.6: Dispiegamento iniziale della flotta (0').

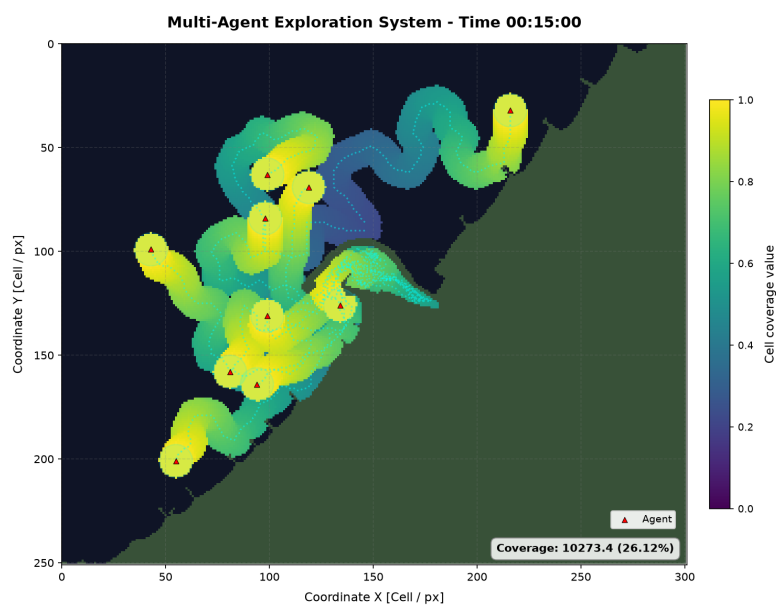


Figura 4.7: Fase di espansione della flotta (15').

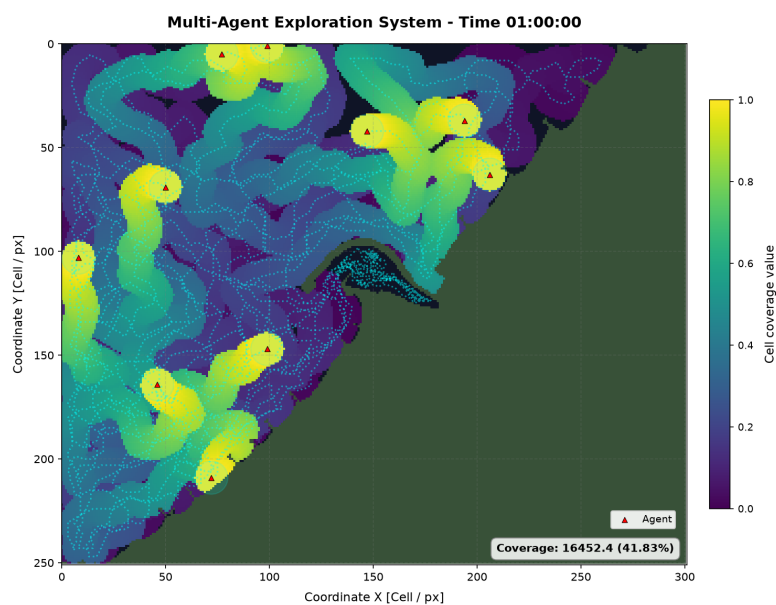


Figura 4.8: Stato finale della flotta (60').

All'avvio della simulazione (Figura 4.6), gli agenti, visivamente rappresentati da triangoli rossi, sono raggruppati all'interno della zona di spawn

circolare tratteggiata in rosso; le impronte dei loro sensori si sovrappongono in modo significativo, evidenziando un’iniziale ridondanza informativa. Non appena ha inizio lo spostamento, tuttavia, l’attivazione del filtro di prossimità spaziale genera un immediato effetto repulsivo implicito.

Con particolare attenzione, si può notare sul campo l’applicazione dei vincoli geometrici trattati nei capitoli precedenti: il **ray-casting sensoriale**, il quale impedisce agli agenti di impostare al valore massimo di 1.0 le celle marine, che pur rientrando nominalmente nel raggio d’azione del sensore, sono separate dal robot da un lembo di terraferma. Parallelamente, si osserva come le celle di terraferma non subiscano alcuna variazione cromatica, rimanendo escluse dall’accumulo e dal calcolo dei valori di copertura della griglia.

Come visibile nella fase di espansione (Figura 4.7), gli agenti si allontanano reciprocamente per esplorare l’acqua libera, dividendosi spazialmente le aree di competenza in modo autonomo e sparpagliandosi a ventaglio per occupare il bacino costiero disponibile. Questo comportamento prova visivamente che l’algoritmo distribuito permette un coordinamento efficace senza la necessità di un’unità centrale di controllo.

Le traiettorie storiche accumulate in azzurro evidenziano come la ripetizione delle rette di Bresenham generi del tutto spontaneamente percorsi curvilinei complessi, caratterizzati da un’elevata fluidità di movimento. Si evidenzia in particolare il perfetto funzionamento del controllo di collisione `is_path_free`: le traiettorie azzurre dei droni si flettono e si allineano geometricamente ai confini fisici della costa di Cecina e della diga portuale (Figura 4.7). Questo dimostra visivamente che gli agenti non selezionano mai target oltre la terraferma e navigano in totale sicurezza entro i soli confini marini del dominio, rispettando rigorosamente la morfologia del territorio

reale.

Un'ulteriore evidenza qualitativa riguarda il comportamento della flotta in prossimità del litorale, dove si osserva come i droni evitino di addentrarsi nelle insenature più strette o a ridosso dei tratti costieri più frastagliati. Questa è una diretta conseguenza del filtro `is_path_free`, che impedisce di selezionare bersagli la cui traiettoria lineare incrocerebbe la terraferma; la copertura di queste acque costiere viene comunque garantita dal raggio d'azione del sensore.

Al raggiungimento dello stato stazionario di regime (Figura 4.8), l'intera porzione marina della mappa mostra una copertura omogenea, con una presenza di zone marine non esplorate ridotta a rari casi isolati. Il fitto intreccio di linee azzurre dimostra che la flotta pattuglia ciclicamente l'intero golfo: i robot ripassano sulle celle marine prima che il decadimento temporale ne azzeri il valore informativo, confermando visivamente l'esistenza del regime stazionario precedentemente analizzato dal punto di vista matematico.

4.4 Analisi parametrica

Al fine di valutare la robustezza rispetto alle condizioni iniziali, l'adattabilità e i limiti operativi dell'algoritmo proposto, la campagna di simulazione ha previsto una serie di test parametrici condotti secondo il principio di variazione di un singolo parametro alla volta, mantenendo inalterata la configurazione nominale di base definita nel capitolo precedente. Nello specifico, sono stati analizzati l'impatto della dimensione della flotta, l'effetto dei vincoli cinematici e sensoriali, l'influenza della velocità di navigazione e la sensibilità del sistema alle condizioni iniziali di dispiegamento.

4.4.1 Dimensione della flotta

La prima indagine ha riguardato la scalabilità del sistema al variare della consistenza numerica della flotta, testando configurazioni con $N \in \{5, 8, 10, 12, 15, 20\}$ agenti cooperativi. L'obiettivo è quantificare come la disponibilità di risorse influenzi sia il valore di copertura a regime, sia la dinamica temporale di esplorazione.

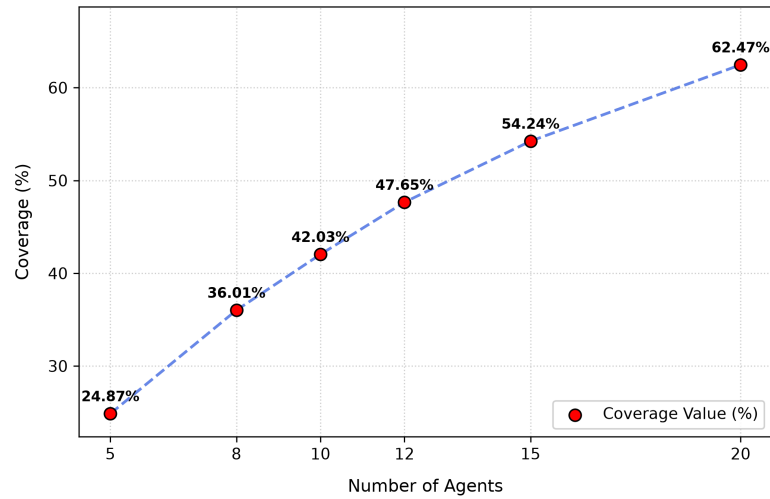


Figura 4.9: Valore di copertura finale al variare del numero di agenti nella flotta.

Come evidenziato in Figura 4.9, la percentuale di copertura finale a regime cresce in modo strettamente monotono all'aumentare del numero di robot, passando dal 24.87% con soli 5 droni fino a raggiungere il 62.47% con una flotta di 20 unità. Tuttavia, la curva manifesta una progressiva deflessione della pendenza, evidenziando chiaramente la classica legge dei **rendimenti marginali decrescenti**, nota in robotica come *Team size effect* [6]:

- l'incremento iniziale da 5 a 10 agenti produce un balzo notevole di copertura (+17.16%), con un apporto medio per singolo drone aggiunto pari a circa il +3.4%;

- al contrario, raddoppiare ulteriormente le risorse passando da 10 a 20 agenti comporta un guadagno complessivo del $+20.44\%$, dimezzando la resa marginale a circa il $+2.0\%$ per drone aggiuntivo.

Questo fenomeno è fisicamente imputabile alla *saturatione spaziale*: all'aumentare della densità dei veicoli all'interno del bacino confinato di Cecina, cresce la probabilità che le aree di influenza dei sensori si sovrappongano o che i droni ripassino su celle recentemente visitate, dove il decadimento non ha ancora ridotto sensibilmente l'informazione.

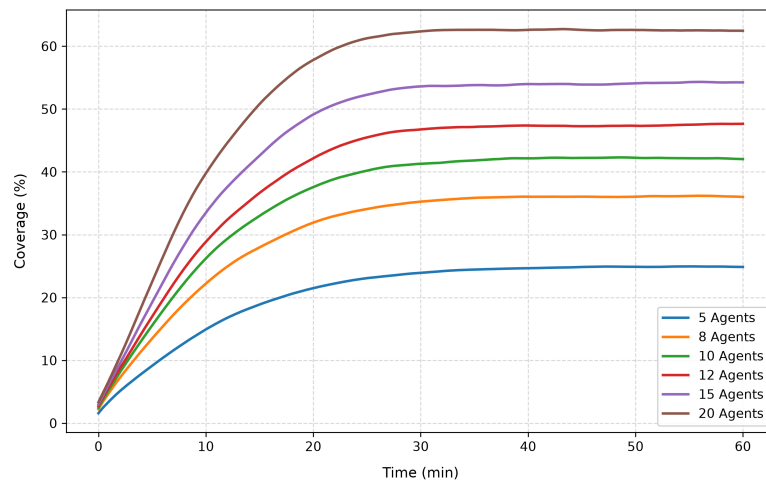


Figura 4.10: Evoluzione temporale della copertura per differenti dimensioni della flotta.

L'analisi dell'evoluzione temporale (Figura 4.10) rivela una proprietà dinamica fondamentale del modello: indipendentemente dal numero di droni impiegati, l'istante temporale in cui il sistema raggiunge lo stato stazionario rimane pressoché identico (attestandosi attorno al minuto 25–30). La dimensione della flotta determina quindi l'ordinata dell'inizio del comportamento asintotico, ma non altera la costante di tempo del sistema, la quale è governata esclusivamente dal tasso di decadimento ambientale e dalla cine-

matica dei vettori. La deviazione standard si mantiene contenuta su tutte le configurazioni, confermando la stabilità statistica dell'approccio.

4.4.2 Campionamento non vincolato

Per validare l'utilità del vincolo geometrico che limita la selezione dei target candidati all'interno della portata del sensore di bordo, è stato condotto uno studio confrontando il modello nominale (*Local sampling*) con una variante priva di tale filtro (*Unbounded sampling*), i cui risultati sono illustrati in Figura 4.11.

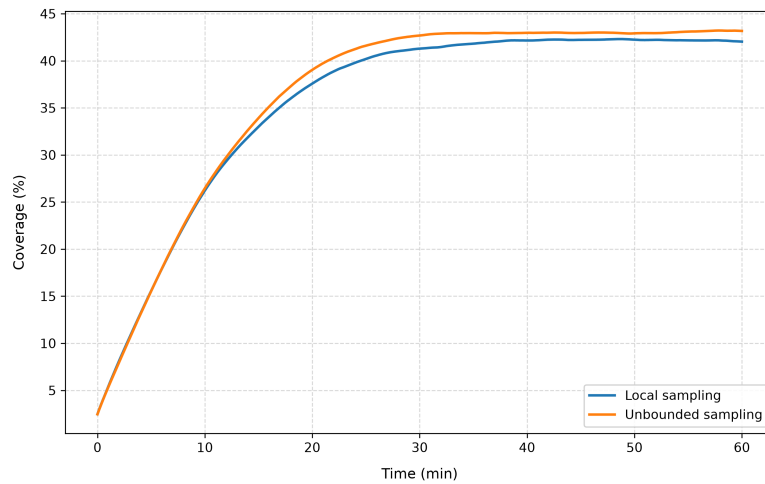


Figura 4.11: Andamento temporale della copertura a confronto tra *Local sampling* e *Unbounded sampling*.

Il campionamento stocastico locale segue una distribuzione normale con deviazione standard pari al raggio del sensore, pertanto, la rimozione del filtro non stravolge la concentrazione dei campioni, ma ammette la possibilità che vengano selezionate coordinate situate nelle code della gaussiana, a distanze superiori alla portata del sensore.

Dal grafico si evince che la rimozione del filtro produce una dinamica di crescita analoga, registrando un incremento marginale del valore di copertura a regime (intorno all'1% , passando da circa il 42% a poco più del 43%). Tale lieve aumento è giustificato dal fatto che gli agenti possono saltuariamente compiere balzi verso celle non ancora visitate e situate al di fuori del campo di visibilità corrente, anticipando l'esplorazione di zone a valore informativo nullo. Nondimeno, l'esiguità di questo guadagno conferma che vincolare la ricerca entro il raggio del sensore è una scelta progettuale ottimale: essa preserva la plausibilità operativa dei veicoli (che si muovono solo verso aree di cui hanno visibilità diretta) senza penalizzare in modo tangibile l'efficienza globale del monitoraggio.

4.4.3 Velocità di navigazione

La velocità nominale degli agenti è stata variata considerando tre regimi operativi distinti: $v_{\text{agent}} \in \{1, 2, 3\}$ m/s. Per garantire la validità scientifica del confronto, il tempo reale simulato è stato mantenuto rigorosamente costante a 1 ora (60 minuti), scalando il numero di iterazioni e i passi temporali conformemente alle relazioni cinematiche definite nel Capitolo 2 ($N_{\text{iterations}} = 360$ per 1 m/s, 720 per 2 m/s e 1080 per 3 m/s).

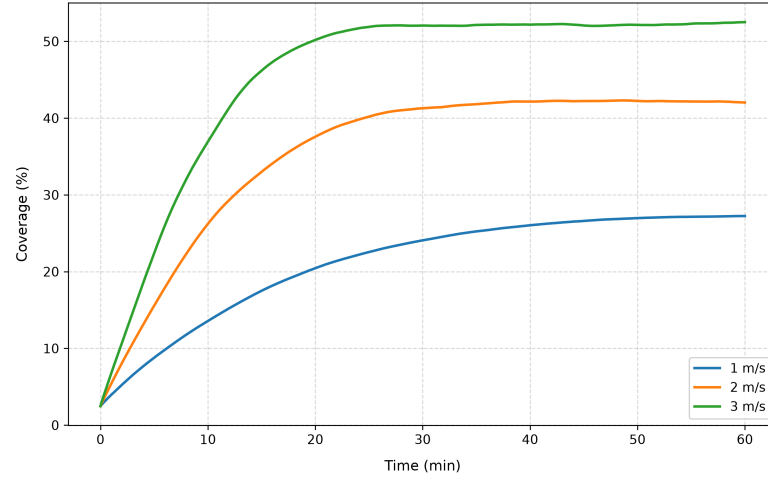


Figura 4.12: Confronto dell'evoluzione temporale della copertura al variare della velocità nominale degli agenti.

I risultati illustrati in Figura 4.12 mostrano come la velocità dei droni eserciti un impatto primario sull'andamento del livello di copertura:

- a velocità ridotta (1 m/s), la flotta copre la mappa molto lentamente, stabilizzandosi su un valore stazionario modesto ($\approx 27\%$): a questa velocità, la frequenza con cui i droni ripassano sulle celle non è sufficiente a contrastare il decadimento, che degrada rapidamente le informazioni raccolte;
- all'aumentare della velocità fino a 3 m/s, la copertura a regime compie un balzo considerevole, superando il 52%. Muovendosi più rapidamente, i droni spazzano un'area superficiale per unità di tempo notevolmente maggiore, acquisendo informazioni sulle celle prima che il decadimento ne abbassi sensibilmente il valore.
- confrontando l'andamento delle curve all'aumentare della velocità si può notare come l'ascissa corrispondente all'inizio del comportamento

asintotico della curva diminuisca; questo è dovuto al fatto che i droni tendono a dispiegarsi in maniera più rapida nell'area costiera.

Si rileva inoltre che a velocità inferiori la deviazione standard finale risulta ulteriormente compressa, poiché il moto più lento riduce la deviazione standard spaziale generata dalle diverse scelte di rotta.

4.4.4 Raggio di spawn

Infine, si è valutata la sensibilità dell'algoritmo alla configurazione iniziale di partenza, confrontando il raggio di spawn nominale ($R_{\text{spawn}} = 20$ celle, pari a un rilascio concentrato di 200 m in prossimità del porto) con una dislocazione fortemente espansa ($R_{\text{spawn}} = 100$ celle, equivalente a una distribuzione su scala chilometrica nel golfo).

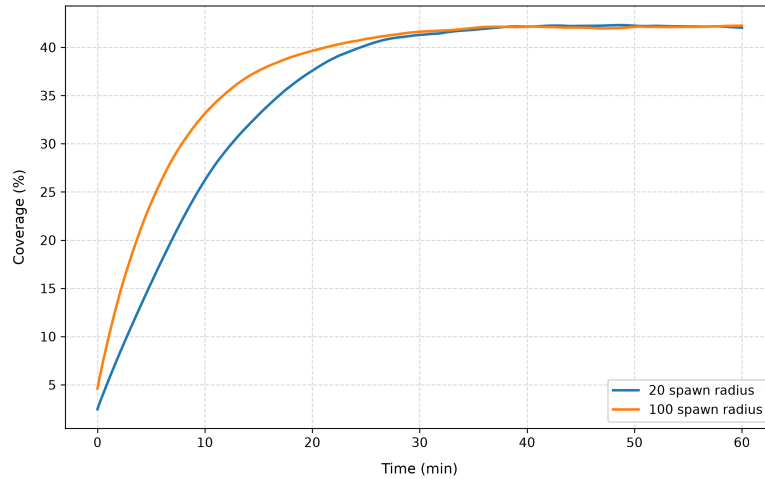


Figura 4.13: Andamento della copertura temporale per differenti raggi dell'area di spawn iniziale.

L'analisi comparativa di Figura 4.13 fornisce una duplice evidenza:

1. **fase transitoria:** con $R = 100$, la copertura parte da un valore iniziale superiore ($\approx 4.6\%$ contro il 2.5% del caso nominale) poiché i sensori dei

droni, partendo già ampiamente distribuiti nello spazio, non subiscono la forte sovrapposizione iniziale tipica del rilascio concentrato in porto. Di conseguenza, la curva relativa a $R = 100$ manifesta una pendenza di salita più ripida nei primi minuti;

2. **convergenza a regime stazionario:** nonostante la marcata differenza iniziale, intorno a metà della simulazione entrambe le curve si sovrappongono perfettamente, attestandosi sul medesimo valore medio di equilibrio ($\approx 42\%$).

Questo risultato costituisce la conferma empirica dell'indipendenza del sistema dalle condizioni iniziali: la flotta, guidata unicamente dalle regole di prossimità spaziale e dalla ricerca del valore di copertura massimo locale, convergerà sempre verso il medesimo valore di copertura, indipendentemente dal fatto che i droni partano ammassati all'interno di una banchina portuale o dispersi al largo in mare aperto.

Capitolo 5

Analisi critica del modello

5.1 Limiti del modello

La scelta progettuale di adottare un approccio geometrico puro ha evidenziato precisi compromessi operativi:

- **vantaggi:** l'algoritmo sviluppato presenta un sovraccarico computazionale estremamente ridotto e non richiede fasi di addestramento o la risoluzione di pesanti modelli fisici a bordo. Questa leggerezza rende il codice altamente portabile e compatibile con la sua esecuzione su microcontrollori commerciali a basso costo;
- **collisioni tra agenti in transito:** sebbene il sistema eviti la collisione con la terraferma e prevenga la sovrapposizione delle destinazioni finali, manca un modulo attivo di evitamento dinamico delle collisioni tra i droni durante la fase di movimento. Tuttavia, data la portata sensoriale contenuta e la presenza del filtro spaziale che impedisce agli agenti di dirigersi verso la stessa identica coordinata target, la probabilità di

collisioni fisiche in mare aperto risulta trascurabile nella pratica, seppur teoricamente non escludibile del tutto;

- **semplificazione del pathfinding:** vista l'impossibilità di selezionare target situati al di là di un ostacolo terrestre (scartati preventivamente per la presenza di terra lungo la linea di Bresenham), gli agenti non necessitano di complessi calcoli per aggirare le barriere geografiche. Sebbene questa scelta escluda localmente i punti situati dietro ai lembi di terra, la natura distribuita della flotta consente ad altri droni, posizionati in punti diversi del bacino, di avere una linea di vista libera verso quelle stesse celle, o allo stesso agente di raggiungerle successivamente da un'angolazione differente. I risultati sperimentali dimostrano che questa apparente limitazione non penalizza la copertura globale del sistema (che si stabilizza comunque su valori soddisfacenti), consentendo al contempo di abbattere l'overhead computazionale di bordo associato ad algoritmi di *path-finding* più complessi;
- **semplificazione cinematica dello spostamento:** il modello della mappa approssima la distanza dello spostamento diagonale tra due celle facendola coincidere con la distanza ortogonale. Questa assunzione, pur restituendo risultati di simulazione validi, allontana leggermente la cinematica virtuale dal dispiegamento reale: procedendo in diagonale, infatti, i droni virtuali viaggerebbero a una velocità effettiva lievemente superiore rispetto a quella nominale impostata. Tuttavia, il vantaggio di questa semplificazione (che consente di eseguire lo spostamento esatto di un pixel per ogni iterazione) risiede nella drastica accelerazione dei tempi di calcolo dell'intera simulazione. Un'alternativa computazionalmente altrettanto valida consisterebbe nell'impedire i movimenti

diagonali, vincolando gli agenti a spostamenti esclusivamente orizzontali e verticali (metrica di Manhattan); tuttavia, tale scelta avrebbe presentato il medesimo problema di discrepanza dal movimento reale (il quale in mare avviene liberamente in ogni direzione), seppur valutato da un punto di vista geometrico diverso.

5.2 Sviluppi futuri

Sulla base dei risultati ottenuti dal simulatore proposto, è possibile delineare alcune estensioni software e algoritmiche:

1. **modularizzazione dell'agente:** si prevede una più netta divisione dei compiti all'interno dell'architettura software, scorporando la logica di calcolo dalla classe **Agent** e strutturandola in una classe **Controller** esterna da iniettare successivamente nel robot. Questa soluzione (riconducibile al design pattern *Strategy*) favorirebbe il confronto diretto tra diverse strategie di esplorazione, permettendo di testare sullo stesso scenario sia l'algoritmo geometrico proposto sia modelli alternativi complessi;
2. **evitamento dinamico delle collisioni:** attualmente non è implementato nel codice alcun algoritmo che impedisca a due droni in stato di **MOVING** di collidere qualora le loro traiettorie rettilinee si incrocino nel medesimo istante temporale. Per superare questa limitazione geometrica durante il moto, si prospetta l'adozione di modelli matematici di sicurezza attiva, quali le *Control Barrier Functions* (CBF) [7] investigate in HYDRAS;

3. **gestione delle parità di copertura nel campionamento:** nel codice attuale, qualora più campioni restituiscano lo stesso identico incremento teorico di copertura, il sistema seleziona il primo target esaminato in ordine di arrivo nell'iterazione. Si potrebbe invece prevedere l'implementazione di una logica di risoluzione delle parità che prediligga il target spazialmente più vicino all'agente: questa ottimizzazione ridurrebbe il tempo di percorrenza della traiettoria e, di conseguenza, minimizzerebbe il periodo di “cecità” decisionale in cui il robot si trova durante lo stato di **MOVING**;
4. **integrazione cinematica continua:** al fine di ottenere una simulazione che corrisponda sempre più fedelmente al reale dispiegamento cinematico degli agenti in mare, si potrebbe prevedere un superamento dello spostamento discreto. Assegnando ad ogni iterazione un valore temporale base di 1 secondo, il drone calcolerebbe un vettore di spostamento continuo (basato sulla velocità nominale e sulla traiettoria di Bresenham), sommando tale grandezza alla propria posizione float. Lo scatto fisico sulla cella successiva della griglia si verificherebbe solo nel momento in cui l'accumulo delle coordinate continue eccede il limite intero del pixel. Tale raffinamento incrementerebbe notevolmente il numero di iterazioni necessarie a coprire lo stesso arco temporale. In tal caso la maggiore fedeltà geometrica del movimento va a discapito del tempo di calcolo computazionale.

Capitolo 6

Conclusioni

Il lavoro di ricerca svolto in questo elaborato di tesi ha condotto alla formalizzazione, allo sviluppo e al testing di un simulatore di un sistema multi-agente decentralizzato, specificamente progettato per il monitoraggio continuo di aree marine costiere tramite una flotta di robot autonomi. L'architettura software proposta, interamente realizzata in Python, si è dimostrata uno strumento computazionalmente efficiente e statisticamente coerente, capace di descrivere in modo affidabile dinamiche esplorative complesse basate su dati cartografici reali.

La campagna di sperimentazione, eseguita sulla mappa reale di Cecina tramite un'analisi statistica condotta su 100 run indipendenti, ha confermato la bontà dell'algoritmo geometrico decentralizzato implementato. Senza alcun bisogno di un'unità centrale di coordinamento, gli agenti si sono distribuiti in modo autonomo ed efficiente sulla griglia marina. L'applicazione del filtro di prossimità spaziale, basato sulle distanze relative dei droni dai target candidati, ha permesso di mitigare preventivamente i conflitti di allocazione spaziale dei target e ha favorito un rapido dispiegamento dei droni nell'area geografica, consentendo al sistema di convergere velocemente verso

valori ottimali di copertura di regime.

Dal punto di vista temporale, lo studio ha dimostrato la convergenza stabile del sistema verso un regime di equilibrio dinamico, con una percentuale di copertura media che si assesta stabilmente intorno al 42% della mappa complessiva. A questo regime stazionario, la frequenza di scansione sensoriale dei 10 droni compensa e bilancia in modo esatto la perdita informativa indotta dal decadimento temporale impostato sulla griglia. La stabilità statistica delle prove è confermata da una deviazione standard estremamente ridotta al regime finale ($\approx 1.19\%$), la quale attesta la robustezza e la ripetibilità dell'algoritmo rispetto allo spawn casuale della flotta, dimostrando come le prestazioni del sistema siano del tutto indipendenti dalle condizioni di dislocazione iniziale. Questo ridotto valore di dispersione dimostra come la componente stocastica legata al campionamento locale dei target non sbilanci le prestazioni delle singole esecuzioni, avvalorando ulteriormente l'affidabilità della logica di controllo sviluppata.

Sotto l'aspetto spaziale, l'analisi degli istogrammi di frequenza distribuiti nel tempo ha provato che oltre il 75% delle celle d'acqua viene mantenuto costantemente in una fascia informativa ottimale (compresa tra 0.4 e 0.5), con una quota minima di celle marine completamente trascurate prossima allo zero (0.1%). Infine, l'integrazione del controllo di collisione basato sull'algoritmo di Bresenham (`is_path_free`) ha garantito una navigazione in totale sicurezza, forzando le traiettorie lineari dei droni a flettersi e ad allinearsi perfettamente ai confini della costa e della diga portuale di Cecina.

In conclusione, quanto sviluppato dimostra che l'algoritmo decentralizzato implementato rappresenta una soluzione concreta, efficace e computazionalmente efficiente per il pattugliamento di ambienti marini complessi. Inoltre, sulla base dei risultati ottenuti, questo approccio si pone come un'ot-

tima fase preliminare di esplorazione e raccolta di informazioni sull'ambiente e sulle potenziali sorgenti inquinanti, prima del dispiegamento delle risorse e delle strategie studiate nel progetto HYDRAS.

Bibliografia

- [1] M. Vergassola, E. Villermanx, and B. I. Shraiman, “‘infotaxis’ as a strategy for searching without gradients,” *Nature*, vol. 445, no. 7126, pp. 406–409, 2007.
- [2] B. P. Duisterhof, S. Li, J. Burgués, V. J. Redondo, G. C. H. E. de Croon, and S. Marco, “Sniffy bug: A fully autonomous swarm of gas-seeking nano quadcopters,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4882–4889, IEEE, 2021.
- [3] J. A. Farrell, S. Pang, and W. Li, “Chemical plume tracing via an autonomous underwater vehicle,” *IEEE Journal of Oceanic Engineering*, vol. 30, no. 2, pp. 428–442, 2005.
- [4] W. Li, J. A. Farrell, and S. Pang, “Maneuvering an autonomous surface vehicle to track chemical plumes,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4216–4221, IEEE, 2006.
- [5] J. E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.
- [6] K. Lerman and A. Galstyan, “Mathematical model of foraging in a group of robots: effect of interference,” *Autonomous Robots*, vol. 13, no. 2, pp. 127–141, 2002.

-
- [7] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.